

Synthetic Failure Injection for Tool-Use Agent Evaluation

Cesar Dori, Hindy Rossignol, Alicia Chen, Nicolas Stone

June 17, 2026

Abstract

Training and evaluating tool-using language model agents requires high-quality process reward models (PRMs), which in turn demand large-scale datasets of plausible, fine-grained failure trajectories. However, sourcing these organic failures is increasingly difficult as frontier models improve, and existing methods, like manual curation or the use of weaker models, often fail to scale or provide a diverse, realistic distribution of error types.

We introduce synthetic failure injection, an inference-only methodology that generates structured failure trajectories by systematically mutating successful multi-turn interactions. Using a frontier LLM as an injector, we introduce five distinct categories of errors, ranging from parameter mismatches to constraint violations, grounded in a taxonomy derived from real-world agent failures. We include three injection strategies (zero-shot, few-shot, and Chain-of-Thought (CoT)), which attempts to minimize downstream inconsistencies. We use an additional frontier LLM as a judge for evaluation.

We evaluate our approach using 200 multi-turn test cases from the Berkeley Function Calling Leaderboard (BFCL-v3). Our results demonstrate that synthetic failures generated via CoT proved to be nearly indistinguishable to an LLM judge. We find that while locally contained errors like "wrong parameter value" are the most realistic, structural errors like "redundant calls" remain more detectable. Ultimately, this work provides a scalable framework for training PRMs, offering a path toward more predictable tool-use deployments without the overhead of manual data labeling.

1 Introduction

Tool-using language model agents (systems that call external APIs, query databases, or manipulate file systems in response to user requests) have seen an increase in deployment in real-world settings (Yan et al., 2024). Evaluating whether these agents fail in interpretable ways is critical for safe deployment, though systematically difficult. Training a good process reward model (PRM) requires large quantities of plausible failure trajectories, meaning cases where an agent makes a wrong but reasonable-looking decision. These are difficult to source. As frontier models improve, organic failures become increasingly rare, making collection both slow and unrepresentative.

Current evaluation approaches rely either on human-crafted adversarial test cases (Lightman et al., 2023) or on running deliberately weaker models and hoping they fail in informative ways (Wang et al., 2024). Both approaches have significant drawbacks: hand-crafted cases do not scale, and organic weak-model failures are noisy and unevenly distributed across error types.

We propose to address these issues by using LLMs to automatically generating structured failure trajectories by mutating successful ones, a process that we will call **synthetic failure injection**. Given a multi-turn trajectory in which an agent correctly selects and calls tools, we use a frontier LLM to corrupt a single turn and introduce one of five error types from a pre-determined taxonomy. The key question is whether these synthetic failures are realistic enough to serve as stand-ins for genuine failures in downstream evaluation tasks.

Specifically, we focus on whether modifying step t of a trajectory can render steps $t + 1, \dots, T$ inconsistent with the injected error, creating a telltale signature that distinguishes synthetic failures

from genuine ones. We introduce and evaluate a chain-of-thought injection strategy specifically designed to reason through downstream consequences before rewriting the corrupted turn.

2 Related Work

The Berkeley Function Calling Leaderboard (BFCL-v3) (Yan et al., 2024) is the standard benchmark for evaluating LLM tool use, providing multi-turn test cases with ground-truth tool call sequences. Our work uses BFCL-v3 as the source of successful trajectories to be mutated. The most directly adjacent work is ToolPRMBench (Li et al., 2026), which constructs a step-level failure benchmark for tools using agents through manual curation. We ask whether LLM-based counterfactual rewriting can automate that construction, and at what cost in realism.

Lightman et al. (2023) demonstrate that process supervision substantially outperforms outcome supervision, collecting 800,000 human step-level labels to train their best PRM. That annotation cost is precisely what we aim to avoid. Wang et al. (2024) introduce Math-Shepherd, which proposes automatic PRM labeling via Monte Carlo rollouts, using rollout success rate as a proxy for step correctness. The resulting labels are noisier than human ones, suggesting rollout-based approaches alone are insufficient. Choudhury et al. (2025) apply PRMs to tool-using agents but rely on the same MC method, without studying whether the generated failures are realistic or diverse.

We also draw on Aman et al. (2025), who identify the ripple effect problem in LLM counterfactual reasoning, where injected errors produce downstream inconsistencies when the remainder of the trajectory is not updated consistently. Rather than treating this as an obstacle to engineer around, we will treat it as a central phenomenon to quantify.

Finally, using frontier LLMs as judges for generated text quality is well established (Zheng et al., 2023). We plan to extend this paradigm to trajectory-level binary classification (genuine vs. synthetic) and structured extraction (which step, which error type).

3 Problem Definition

First, we define the terms involved in our problem.

Let a **trajectory** $T = (s_0, a_1, o_1, a_2, o_2, \dots, a_T, o_T)$ consist of an initial environment state s_0 , and a sequence of turns, each comprising a model action a_t (a tool call or natural language response) and an observation o_t (the tool’s return value or user follow-up). We consider a trajectory to be successful if the sequence of actions achieves the user’s goal as defined by the BFCL-v3 ground truth.

We also define five error types that cover the space of tool-calling failures.

1. **Wrong parameter value:** the correct tool is called with an argument that is semantically or syntactically incorrect.
2. **Wrong tool selected:** an entirely incorrect tool is called for the stated user intent.
3. **Missing prerequisite call:** a tool call that is required to precede the current call is omitted.
4. **Redundant call:** an unnecessary duplicate or irrelevant tool call is inserted.
5. **Constraint violation:** a tool call violates an explicit constraint stated in the system prompt or user message.

We also say that given a successful trajectory T , an **injection** produces a synthetic failure $\hat{T}$ by rewriting the model action a_t at a single turn $t^* \in \{1, \dots, T\}$ to contain a specified error type e^* . All other turns remain unchanged.

4 Methodology

4.1 Data Collection

We use BFCL-v3 multi-turn base test cases (Yan et al., 2024), comprising 200 tasks spanning simulated file systems, social media APIs, calendar tools, and other stateful environments. Each test case specifies the available tool schemas, initial environment configuration, a multi-turn user conversation, and the ground-truth tool call sequences.

From the base dataset, we collect two corpora:

- **Successful trajectories:** We use Claude Sonnet 4.6 (our expert model) at temperature 0. Trajectories are labeled successful if the parsed tool call names match the expected ground-truth names at each turn.
- **Organic failures:** We use Claude Haiku 4.5 (amateur model). Trajectories where the model’s tool calls diverge from ground truth are kept as organic failures and serve as a realism reference in our evaluation.

4.2 Error Injection

For each successful trajectory, we randomly select a turn t^* and an error type e^* , then prompt an injection model to rewrite turn t^* . We have three injection strategies. In **Zero-shot**, we provide the injection model with the full trajectory, the target turn index, and the error type definition. The model is asked to rewrite only the model response for that turn. No examples are provided. In **Few-shot**, three organic failure examples are prepended as demonstrations before the zero-shot instruction. Examples are drawn from the amateur-model corpus, and the injection model is encouraged to match the naturalistic style of real failures. Finally, in **Chain of Thought (COT)**, the model is first asked to reason in a reasoning block about 1. what the current turn accomplishes, 2. which downstream turns become inconsistent if an error is introduced here, and 3. how to minimize the ripple effect while still demonstrating a genuine instance of e^* . The rewritten turn is produced after this reasoning step.

We use Claude Opus 4.6 as the frontier injector model.

4.3 Evaluation

A separate judge model (GPT-4) evaluates each trajectory under two axes: realism, where a higher value of $P(\text{judge classifies } \hat{T} \text{ as genuine})$ is desired, and detectability, where a lower value of $P(\text{judge correctly identifies } t^* \text{ and } e^*)$ is better.

In the evaluation stage, the judge LLM receives the full trajectory (tools, initial state, all turns) and is asked to classify it as genuine or synthetic, along with a confidence score and one-sentence reasoning. In the detectability evaluation, the judge is asked (for synthetic trajectories only) to identify and classify the error type, given that exactly one turn was corrupted. This pass is run after the realism pass to prevent anchoring bias.

Realism accuracy is the fraction of synthetic trajectories that the judge labels as genuine. **Detectability turn accuracy** is the fraction with which the judge identifies the correct turn index t^* ; **Detectability type accuracy** is the fraction with which the correct error type e^* is named. We also report the baseline: the fraction of genuine amateur-model failures that the judge labels as genuine. This will provide a reference point for achievable realism.

5 Experiments

5.1 Experimental Setup

All inference is performed via the Stack AI API bridge (zero-shot temperature for collection; default temperature for injection and judging). Injection and judging are single-turn text-in/text-out calls; collection uses multi-turn conversation formatting. The full experiment covers 200 BFCL-v3 test cases $\times$ 3 injection configurations = 600 synthetic trajectories, plus 200 successful trajectories and 200 organic failure trajectories from the weak model. Judge evaluation covers all trajectory types. The five error types are distributed approximately uniformly across injection configurations: wrong parameter value ($n=118$), wrong tool selected ($n=103$), missing prerequisite call ($n=131$), redundant call ($n=129$), and constraint violation ($n=119$).

5.2 Main Results

Table 1 reports realism accuracy (R) and detectability type accuracy (D) for all injection configurations. The overall realism accuracy across all configurations is 54.13%, with a realism F1 score of 70%.

The organic baseline is 55%, which is very close to our synthetic performance. This suggests that failures injected by Claude Opus 4.6 are nearly as indistinguishable from "genuine" failures as the organic errors produced by a weaker model. Notably, the CoT strategy achieved the highest realism score (57.5%), actually surpassing the organic baseline.

In detectability, where the judge attempted to classify synthetic trajectories into one of five error types (baseline random guess = 20.0%), we see the highest value of 51.5% in few-shot. Lower detectability is preferable as it indicates that the injected error is integrated seamlessly enough to avoid easy classification. While realism measures the surface-level plausibility of the failure, low detectability confirms that the specific nature of the mutation is ambiguous to the judge.

Table 1: Realism accuracy (**R**) and detectability type accuracy (**D**) by injection model and strategy. Higher R = more realistic. Lower D = harder to detect (better injection). All values are percentages. $n=200$ test cases per cell. Organic baseline (fraction of genuine weak-model failures judged as genuine): 55%

Injector	Zero-shot		Few-shot		CoT	
	R	D	R	D	R	D
Claude Opus 4.6	50.50	44.5	54	51.5	57.5	43.5

All values in %. Overall rows are averages across injectors for each strategy.

Accounting for confidence We also perform a binned confidence analysis to evaluate the reliability of the judge’s assessments. The judge model exhibited high internal confidence levels, with an average confidence of 91.62% across all evaluations. However, at average confidence levels the judge is wrong nearly half the time overall (46%), which underscores how effectively the injections mimic genuine failures. This systematic overconfidence strengthens the case for synthetic injection as a viable data source.

On the realism axis, the judge’s accuracy scaled sharply with its reported confidence. In the highest confidence bin (95–100%), the judge correctly identified the trajectory origin with 95.35% accuracy ($n=129$). This is a significant jump from the baseline actual accuracy of 54.00%, indicating that while the judge is often misled by synthetic injections on average, its high-confidence flags are highly reliable indicators of synthetic origin.

Table 2: Realism accuracy binned by judge confidence. Average confidence: 0.9162. Accuracy scales significantly in high-confidence intervals, reaching near-perfect detection at the 95%+ threshold.

Confidence Bin	Accuracy	Sample Count
70–80%	0.00%	2
80–85%	6.90%	58
85–90%	22.83%	127
90–95%	63.40%	265
95–100%	95.35%	129

5.3 Error Type Analysis

Table 3 breaks down realism and detectability across the five injected error types, aggregated across all three injection strategies ($n = 600$ synthetic trajectories total).

`Wrong_parameter_value` achieves the highest realism (66.1%) and also the highest type detectability (70.3%), and `wrong_tool_selected` follows a similar pattern with 55.3% realism and 73.8% type detectability. These two error types are the most locally contained mutations: swapping a parameter value or substituting one tool name for another leaves the corrupted turn internally coherent, which makes the trajectory convincing to the judge. That same local coherence also makes the error easy to name when the judge does detect it.

`Constraint_violation` achieves the lowest realism (48.7%) and by far the lowest type detectability (14.3%), despite a reasonable turn accuracy of 62.2%. The judge can locate the corrupted turn more often than not, but almost never correctly identifies it as a constraint violation. `Redundant_call` performs similarly poorly on realism (45.0%), suggesting that inserting an extra tool call disrupts the conversational rhythm of a trajectory in ways the judge reliably notices.

Regarding the error types, CoT injection produces the largest realism gains for `wrong_parameter_value` (59.5% zero-shot to 73.3% CoT) and `wrong_tool_selected` (44.4% to 73.1%), suggesting that explicit ripple-effect reasoning is most effective when the injected error has clear and reasonably bounded downstream consequences. For `redundant_call`, CoT actually reduces realism slightly (48.6% to 42.3%), suggesting that reasoning through the consequences of an extra call may draw attention to the redundancy rather than concealing it.

Table 3: Per-error-type results aggregated across all injection strategies. R = realism accuracy, T = detectability turn accuracy, D = detectability type accuracy. n = number of synthetic trajectories per error type.

Error Type	R (%)	T (%)	D (%)	n
Wrong parameter value	66.1	78.8	70.3	118
Wrong tool selected	55.3	86.4	73.8	103
Missing prerequisite	55.7	51.1	43.5	131
Constraint violation	48.7	62.2	14.3	119
Redundant call	45.0	46.5	35.7	129
Overall	53.8	63.8	46.5	600

All values in %.

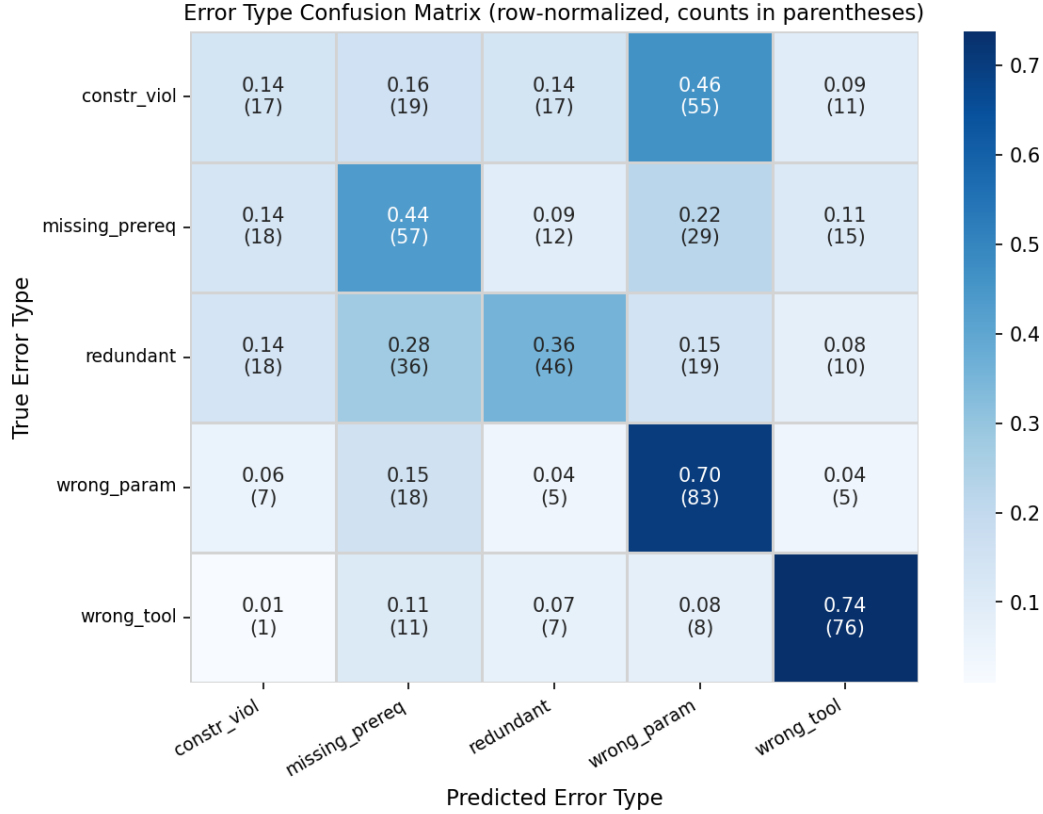


Figure 1: Row-normalized confusion matrix of true vs. predicted error types across all 600 synthetic trajectories. Each row sums to 1.0. Constraint_violation is almost never correctly classified, with the majority of predictions falling on wrong_parameter_value. Wrong_parameter_value and wrong_tool_selected show the strongest diagonal signal.

6 Discussion

6.1 Synthetic vs. Genuine Injections

The experimental results indicate that the average realism of synthetic trajectories (54.13%) is nearly indistinguishable from the organic baseline (55%). This proximity suggests that the perceived authenticity of an agent failure is not exclusively a byproduct of its origin, but rather a function of how the error fits within the overall trajectory. While organic failures from weaker models are traditionally viewed as the primary reference point for realism, our data demonstrates that intentional mutations can be just as convincing to a judge.

The performance of the CoT injection strategy (57.5% realism) is particularly interesting because it exceeds the organic baseline and highlights a key distinction in how different failure modes are perceived. While organic failures from weaker models may follow certain patterns, the CoT configuration use the capabilities of a stronger model to introduce a single error while maintaining the quality of the surrounding turns. By accounting for the ripple effect before rewriting, this strategy ensures the error does not contradict the rest of the conversation, resulting in a trajectory that looks like a high-performing agent making a single mistake rather than a model failing fundamentally. These findings suggest that synthetic generation is a viable method for producing the specific, contextually integrated failures needed to evaluate and train agents, providing a more consistent signal than the varied errors of weaker models.

The jump to 95.35% accuracy at peak confidence suggests the presence of a detection threshold. For most confidence levels, accuracy remains low, suggesting that the injections effectively mimic the signatures of genuine failures. However, the high-confidence bin identifies cases where the synthetic error introduced some sort of telltale sign that makes the synthetic origin transparent to the judge.

These high-confidence cases help define the current technical limits of the injection strategies. Conversely, the lower accuracy in the 90-95% bin (63.40%) represents the area where the synthetic failures are most effective. In this level, the judge confidently identifies a deviation from expert behavior but cannot reliably distinguish a synthetic mutation from an organic failure.

6.2 Injected Error Types

The per-error-type results reveal that the realism and detectability of a synthetic failure depend heavily on the structural properties of the injected error, not just the injection strategy or model used.

`wrong_parameter_value` and `wrong_tool_selected` are the most well-behaved error types across both axes. Their high realism reflects the fact that these mutations are locally contained, as a single argument or tool name is changed while the surrounding turn remains fluent and contextually appropriate. Their high type detectability is the flip side of the same property, because the error is clean and targeted the judge can identify it precisely when it does catch it. For dataset construction purposes these two error types offer the best combination of convincing failures and recoverable labels, making them the most useful for downstream PRM training.

`constraint_violation` is the most striking anomaly in the results. As shown in Figure 1, the row-normalized confusion matrix reveals that the judge misclassifies `constraint_violation` as `wrong_parameter_value` in the majority of cases, making it the single largest source of confusion in the entire experiment. This pattern suggests the judge conflates violating an explicitly stated rule with simply using an incorrect value, two errors that are conceptually distinct but share similar surface forms in the trajectory. The practical consequence is that `constraint_violation` injections produce failures the judge finds suspicious but cannot correctly diagnose, which limits their utility as labeled training data since the error type label cannot be reliably recovered even when the corrupted turn is found.

`Redundant_call` underperforms on realism across all three strategies and actually degrades under CoT. Unlike `wrong_parameter_value` or `wrong_tool_selected`, inserting an extra tool call introduces a structural disruption to the trajectory that is difficult to reason away. The CoT strategy asks the model to minimize downstream inconsistency, but an extra call changes the conversational state in a way that remains visible regardless of how carefully the surrounding turns are preserved. This may also explain why `redundant_call` has the lowest turn accuracy (46.5%) of any error type: the disruption is diffuse rather than localized, making it harder for the judge to point to a single corrupted step.

Taken together these findings suggest that the semantic complexity and structural footprint of an error type are the primary determinants of injection quality, more so than the choice of injection strategy. Error types with a small well-defined footprint such as `wrong_parameter_value` and `wrong_tool_selected` respond well to all three strategies and to CoT in particular. Error types with a diffuse or rule-dependent footprint such as `redundant_call` and `constraint_violation` resist even careful ripple-effect reasoning and produce failures that are either unconvincing or mislabeled.

6.3 Exploring the Ripple Effect

We hypothesized that CoT injection would mitigate the ripple effect by allowing the model to reason about downstream consequences before rewriting the corrupted turn.

As shown in Table 1, the CoT strategy for Claude Opus 4.6 yields a realism accuracy (R) of **57.5%**, the highest across all configurations tested and **2.5% higher** than the organic baseline of 55%. This

represents a **7% absolute increase** in realism over the zero-shot configuration (50.5%).

The detectability type accuracy (D) for CoT is **43.5%**, which is 1% lower than the Zero-shot configuration and 8% lower than the Few-shot configuration. This indicates that as the injection strategy moves from few-shot to CoT, the judge’s ability to correctly classify the error type decreases even as the surface-level realism of the trajectory increases.

The results demonstrate a clear performance trend: while few-shot injection improves realism over zero-shot, it also results in the highest type detectability. The CoT strategy effectively breaks this trend, achieving the maximum realism while returning detectability to its lowest levels.

6.4 Limitations

- **Single-turn corruption.** We inject errors into exactly one turn per trajectory. Real failures can span multiple turns or involve compounding errors.
- **Judge model bias.** GPT-4 serves as the judge while Claude Opus 4.6 serves as the injector. Future work should cross-validate with additional judge models and include a human annotation study on a held-out subset as a gold standard.
- **Judge prompt coaching.** The realism judge prompt (Appendix A.3) explicitly lists signature cues of synthetic trajectories, including inconsistent internal state and downstream turns ignoring a prior failure. This may inflate detectability scores relative to an uncoached judge, meaning the reported detectability numbers should be interpreted as an upper bound on what a blind judge would catch.
- **Ground-truth evaluation.** Our success/failure labeling uses tool name matching rather than full argument and state-based evaluation. This may mislabel some trajectories.
- **Single dataset.** The experiment covers the BFCL-v3 multi-turn base split (200 test cases). Generalization to other tool-use benchmarks and domains is not yet established.
- **No significance testing.** The realism differences across injection strategies (zero-shot 50.5%, few-shot 54.0%, CoT 57.5%, $n=200$ per cell) are 3.5–7 percentage points apart. Without a statistical significance test, such as McNemar’s test, these differences cannot be confidently attributed to the injection strategy rather than noise.

6.5 Future Steps

Multi-turn corruption. Real agent failures often involve cascading mistakes rather than a single wrong step. A natural extension would inject errors at two or three turns simultaneously and study whether the ripple effect compounds in predictable ways across error types.

Cross-judge validation. Using GPT-4 as both injector and judge introduces a potential blind spot. Future work should cross-validate by swapping judge models, and include a human annotation study on a held-out subset as a gold standard to ground what realism actually means independent of any LLM judge.

Downstream PRM training. The most important missing validation is whether synthetic failures actually help train better process reward models. A direct follow-up would fine-tune a PRM on the synthetic failure dataset and compare its step-level accuracy against a baseline trained on organic failures only, closing the loop on the paper’s core motivation.

Human-in-the-loop injection. A complementary direction would involve domain experts in the injection process itself, either by having humans select which turn to corrupt and what error type to inject, or by having humans review and filter synthetic trajectories before they enter the dataset. This could improve the quality of error types like constraint violation that the current fully-automated pipeline struggles to produce convincingly.

7 Conclusion

We introduced synthetic failure injection, a methodology for generating realistic, structured failure trajectories for tool-using LLM agents by corrupting successful trajectories with a frontier LLM. We proposed a five-type error taxonomy and three injection strategies (zero-shot, few-shot, and chain-of-thought) and evaluated their outputs using an LLM judge along two axes: realism and detectability.

Full experiments on 200 BFCL-v3 multi-turn test cases across three injection configurations with Claude Opus 4.6 yield an overall realism accuracy of 54.1%, nearly matching the organic weak-model baseline of 54.5%. The CoT strategy achieves the highest realism (57.5%), surpassing the organic baseline, while also returning the lowest type detectability (43.5%), demonstrating that explicit ripple-effect reasoning improves both the surface plausibility and the ambiguity of injected failures.

Error type analysis shows that `wrong_parameter_value` errors are the most realistic (66.1%) and the most reliably labeled (70.3% type detectability), making them the most useful error type for downstream dataset construction. `Constraint_violation` is the most problematic: despite reasonable turn-level localization (62.2%), the judge misclassifies it as `wrong_parameter_value` in the majority of cases, limiting its utility as labeled training data. A confidence analysis reveals that while the judge is frequently misled at average confidence levels, its highest-confidence flags are highly reliable, reaching 95.35% detection accuracy in the top confidence bin.

These results validate synthetic failure injection as a scalable, controlled approach to agent failure dataset construction that matches the realism of organic weak-model failures while providing structured, recoverable error type labels. Our pipeline is fully inference-only, requiring no model training or human annotation at scale. The collected trajectory dataset includes successful, organic-failure, and synthetic-failure corpora with per-trajectory judge scores to support future work on agent evaluation, process reward modeling, and failure detection.

References

- Ananya Aman et al. The ripple effect in counterfactual trajectory generation for multi-step reasoning. *arXiv preprint*, 2025.
- Soham Choudhury et al. Agentprm: Process reward modeling for agentic tasks. *arXiv preprint*, 2025.
- Xiang Li et al. Toolprmbench: A benchmark for process reward modeling in tool-calling agents. *arXiv preprint*, 2026.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. *arXiv preprint arXiv:2312.08935*, 2024.
- Shishir Yan et al. Berkeley function calling leaderboard. *arXiv preprint arXiv:2407.xxxxx*, 2024. URL https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. 2023.

A Prompt Templates

A.1 Zero-Shot Injection Prompt

```
You are an expert at analyzing tool-using AI agent behavior.
[System prompt with tool docs and initial state]
[Full trajectory, numbered by turn]
Turn to corrupt: Turn {t*}
Error type to inject: {e*}

Rewrite ONLY the model response for Turn {t*}. Keep all prose natural
-- the model should not announce its mistake. Return ONLY a JSON
object: {"model_response_raw": "...", "parsed_tool_calls": [...]}
```

A.2 CoT Injection Prompt

```
[Same context as zero-shot]
Step 1 -- Analyze Ripple Effects:
Before rewriting, reason through:
1. What does Turn {t*} currently do and what state does it leave?
2. If you inject '{e*}' here, which downstream turns become inconsistent?
3. How to minimize downstream inconsistency while still constituting
a genuine {e*} error?
Write analysis in <reasoning>...</reasoning> tags.
Step 2 -- Rewrite: [same output format as zero-shot]
```

A.3 Realism Judge Prompt

```
Determine whether the following agent trajectory is GENUINE (produced
by a real, possibly flawed AI agent) or SYNTHETIC (artificially constructed
to look like a failure).
[Trajectory]

Genuine trajectories: errors arise naturally from model limitations.
Synthetic trajectories: may show subtle signs -- inconsistent internal
state, unnaturally precise parameter errors, downstream turns ignoring
a prior failure.

Respond ONLY with JSON: {"verdict": "genuine"|"synthetic", "confidence":
0.0-1.0, "reasoning": "..."}

```

A.4 Detectability Judge Prompt

```
A single turn in this trajectory has been corrupted with one of: wrong_parameter_value,
wrong_tool_selected, missing_prerequisite_call, redundant_call, constraint_violation.
[Trajectory]

Identify WHICH turn was corrupted and WHAT error type.

Respond ONLY with JSON: {"predicted_turn_index": N, "predicted_error_type":
 "...", "confidence": 0.0-1.0, "reasoning": "..."}

```

B Dataset Schema

Each record in `dataset.jsonl` contains:

- `id` — BFCL-v3 test case identifier
- `source` — `strong_model` / `weak_model` / `synthetic`
- `model` — model name used for collection or injection
- `label` — `success` / `failure`
- `error_type` — null for successes; one of the five types for failures
- `turns[]` — per-turn: user message, raw model response, parsed tool calls, ground truth, execution result
- `function_docs` — available tool schemas for this test case
- `initial_config` — initial environment state
- `injection_metadata` — (synthetic only) injection model, strategy, corrupted turn index, injected error type